

Experiment-5-Advanced-1

Title: FPGA-Based Autonomous Hardware Failure Diagnosis and Fault Localization Engine

Objective:

The objective of this experiment is to design and verify an on-chip hardware engine that can detect faults in a digital circuit and localize them to a specific internal net, without an external tester or a human in the loop. The engine uses scan-based testing: the flip-flops of the circuit under test (CUT) are linked into scan chains, a known pattern is shifted in, the circuit is clocked in functional mode, and the captured response is shifted out and compared bit-by-bit with the fault-free (golden) response. Every mismatch is retained in a fail log rather than compressed into a signature, so that the evidence can be traced back to the cause (effect-cause diagnosis).

The design is described in SystemVerilog (IEEE 1800-2012), verified through ModelSim RTL simulation, and targeted for implementation on the Terasic DE10-Nano (Intel Cyclone V SoC) FPGA board. This experiment covers the first two stages of the engine: Stage 0, which checks the integrity of the scan chains themselves, and Stage 1, which narrows the suspected fault site using structural cone intersection. Fault injection hardware is built into the CUT so that stuck-at, transition and scan-chain faults can be created on demand and the diagnosis result can be checked against a known ground truth.

Block Diagram:

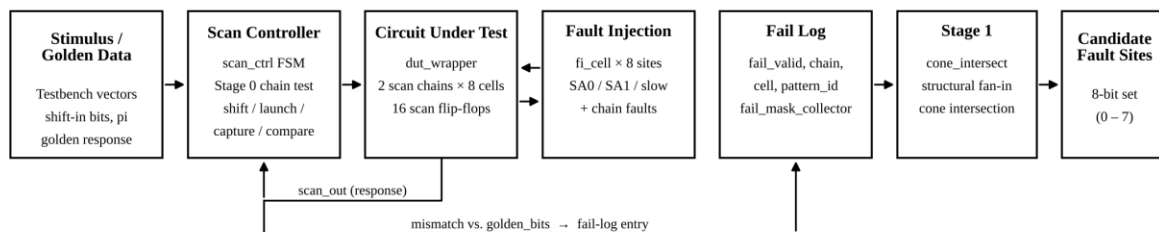


Fig. 1. Overall architecture of the FPGA-based hardware fault diagnosis and fault localization engine (Stage 0 + Stage 1).

The test stimulus (shift-in bits, primary inputs pi) and the expected golden response are supplied to the scan controller. The scan controller first runs a Stage 0 chain test to confirm that every scan chain shifts correctly. It then drives the scan-enable and scan-in signals of the circuit under test, which is a scan-inserted placeholder circuit with 16 flip-flops arranged as two chains of eight cells. Eight fault-injection cells sit on the internal nets of the CUT and can force a stuck-at-0, stuck-at-1 or slow-to-transition fault, while a separate injector can create stuck, broken or slow scan-chain cells. The response shifted out on scan_out is compared with golden_bits every cycle; each mismatch produces a fail-log entry tagged with the pattern, chain and cell. The fail-mask collector gathers these entries into a per-pattern fail mask, and the Stage 1 cone-intersection block intersects the fan-in cones of the failing cells to produce the set of candidate fault sites (0–7).

RTL Design:

The engine is implemented entirely in synthesizable SystemVerilog and is organised as a set of small modules with clear interfaces. Stage 0 is formed by the fi_cell.sv, dut_wrapper.sv, scan_ctrl.sv and stage0_milestone_top.sv modules. Stage 1 adds fail_mask_collector.sv, cone_intersect.sv and stage1_milestone_top.sv on top of the unchanged Stage 0 design. Two self-checking testbenches,

tb_stage0_milestone.sv and tb_stage1_milestone.sv, are used for functional verification. The module hierarchy and code structure are shown in Fig. 2.

1. fi_cell.sv

The fi_cell module is the fault-injection primitive. One instance is placed on each candidate net of the CUT and passes the net value through unchanged unless it is selected. Each instance has a fixed SITE_ID parameter and is selected when fi_active is high and fi_site equals its SITE_ID. The 2-bit fi_type input then chooses the fault: 01 forces the net to 0 (stuck-at-0), 10 forces it to 1 (stuck-at-1), and 11 substitutes a one-clock-delayed copy of the net, which models a slow-to-transition fault. The selection is implemented as a small combinational multiplexer, so it adds no latency when no fault is injected.

2. dut_wrapper.sv

The dut_wrapper module is the circuit under test. It is a hand-designed placeholder with four primary inputs, four primary outputs and 16 scan flip-flops arranged as two scan chains of eight cells (NUM_CHAINS = 2, CHAIN_LEN = 8). Each flip-flop has a 2-to-1 multiplexer in front of it: when scan_en is high, the cell loads the value of the previous cell in its chain (or scan_in for the first cell), and when scan_en is low it loads its functional next-state value. This mux-D structure is the standard scan-insertion scheme. Eight internal nets of the combinational logic are routed through fi_cell instances (NUM_FAULTS = 8). The module also contains a chain-fault injector that can make a selected scan cell stuck, broken or slow, so that faults in the test infrastructure itself can be exercised. The interface is designed so that a scan-inserted ITC'99 b17 benchmark netlist can replace this placeholder later.

3. scan_ctrl.sv

The scan_ctrl module is the test sequencer and the core of Stage 0. It is a finite state machine with the states S_IDLE, S_SHIFT_IN, S_LAUNCH, S_CAPTURE, S_SHIFT_OUT and S_DONE. In chain_test_mode it flushes a repeating 0011 pattern through every chain and compares the output with the input delayed by CHAIN_LEN + 1 cycles; the result is reported per chain on chain_ok, together with the first failing cell on chain_first_bad_cell_flat. In normal mode it shifts the test pattern in with scan_en high, applies one functional capture clock, and shifts the response out while comparing scan_out with golden_bits every cycle. A mismatch raises fail_valid and records fail_pattern_id, fail_chain_id and fail_cell_index. When two_pattern_mode is set, the extra S_LAUNCH state applies a second functional clock, giving the launch-on-capture (V1 → V2) sequence needed to detect transition faults. scan_en is asserted in the same clock edge that leaves S_IDLE so that every shift cycle is a true shift cycle.

4. stage0_milestone_top.sv

The stage0_milestone_top module is the Stage 0 integration level. It contains no logic of its own; it instantiates scan_ctrl and dut_wrapper and connects the scan-enable, scan-in, scan-out and capture signals between them, while bringing the fault-injection controls, test-mode controls and fail-log outputs out to the ports.

5. fail_mask_collector.sv

The fail_mask_collector module converts the serial fail-log stream from scan_ctrl, which reports one mismatching cell per cycle, into a single 16-bit fail_mask per pattern in which each set bit marks a scan cell that captured a wrong value. It also reports whether any cell failed in that pattern. This per-pattern bitmap is the form of evidence that Stage 1 requires.

6. cone_intersect.sv

The cone_intersect module implements Stage 1 fault localization. For every scan cell, a table gives the structural fan-in cone, i.e. the set of fault sites that can logically reach that cell in one functional clock. For each failing pattern, the module takes the intersection of the cones of all failing cells, and across the patterns of a session it intersects these results again, so the 8-bit candidate_sites set can only shrink. A new_session signal resets the set to all ones. Passing cells are deliberately not used to remove candidates, because a fault can be present but masked at a passing cell. Runs made in two_pattern_mode are ignored, since the cone table models a single functional clock. The cone table is hand-derived for the placeholder CUT and will be regenerated for the real benchmark.

7. stage1_milestone_top.sv

The stage1_milestone_top module is the top level of this experiment. It wraps the verified stage0_milestone_top without modifying it and adds the fail_mask_collector and cone_intersect modules, exposing the fail mask and the candidate fault-site set as outputs.

8. Testbench and Verification

Two self-checking testbenches were used for RTL verification. The tb_stage0_milestone.sv testbench first captures the golden response with no fault injected and confirms that a fault-free run produces zero mismatches. It then runs the Stage 0 chain test on healthy chains and on stuck, broken and slow chain cells, and injects stuck-at-0, stuck-at-1 and slow-to-transition faults on the functional nets, checking the number and location of the fail-log entries for each case. The transition fault is run twice: in single-capture mode, where it is expected to be undetectable, and in two-pattern mode, where it must be detected. The tb_stage1_milestone.sv testbench injects a fault at each of the eight sites and checks the candidate set produced by cone intersection, including unique localization, ambiguous sets, a masked fault, multi-pattern sessions, contradictory evidence and the two-pattern guard. Both testbenches were simulated in ModelSim Intel FPGA Starter Edition 10.5b with the -sv option. The Stage 0 testbench reported 8 PASS, 1 EXPECTED and 0 FAIL, and the Stage 1 testbench reported 15 passed and 0 failed.

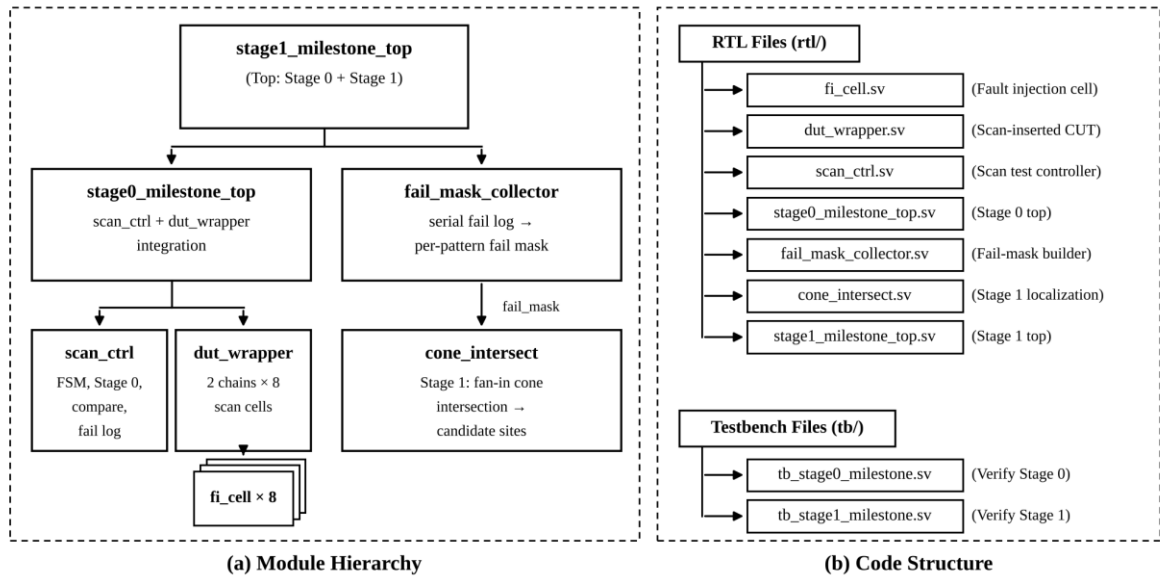
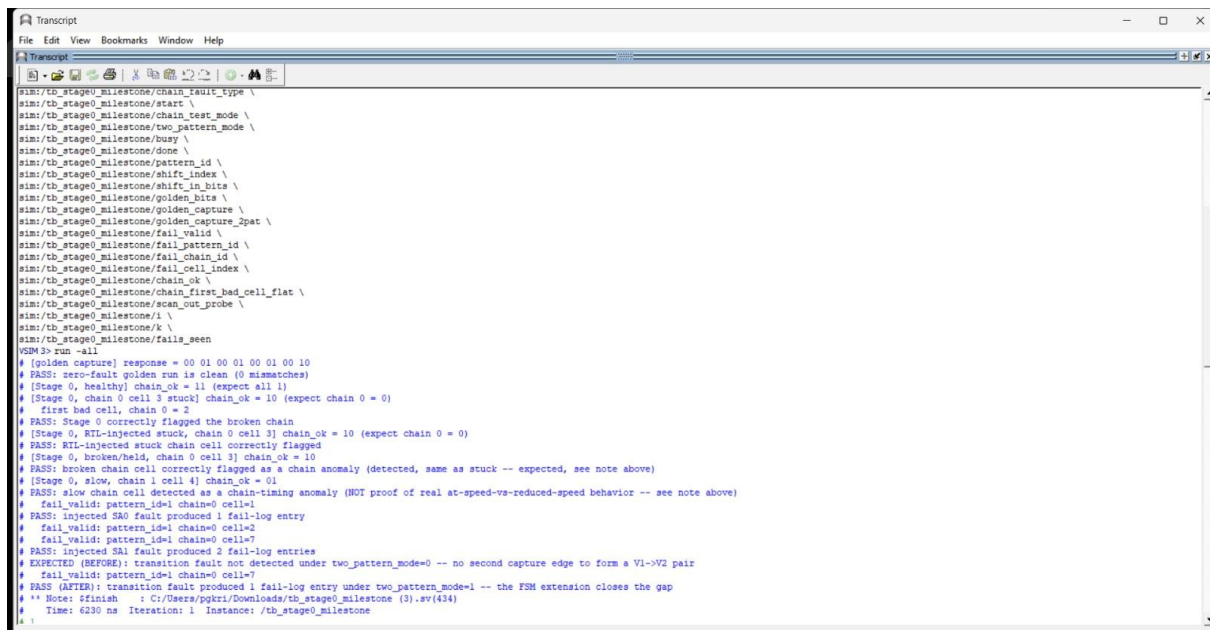


Fig. 2. Module hierarchy and code structure of the fault diagnosis engine.

Simulation Results:



```
Transcript
File Edit View Bookmarks Window Help

sim:/tb_stage0_milestone/chain_fault_type \
sim:/tb_stage0_milestone/start \
sim:/tb_stage0_milestone/chain_test_mode \
sim:/tb_stage0_milestone/two_pattern_mode \
sim:/tb_stage0_milestone/bury \
sim:/tb_stage0_milestone/done \
sim:/tb_stage0_milestone/pattern_id \
sim:/tb_stage0_milestone/shift_index \
sim:/tb_stage0_milestone/shift_in_bits \
sim:/tb_stage0_milestone/golden_bits \
sim:/tb_stage0_milestone/golden_capture \
sim:/tb_stage0_milestone/golden_capture_2pat \
sim:/tb_stage0_milestone/fail_valid \
sim:/tb_stage0_milestone/fail_pattern_id \
sim:/tb_stage0_milestone/fail_chain_id \
sim:/tb_stage0_milestone/fail_cell_index \
sim:/tb_stage0_milestone/chain_ok \
sim:/tb_stage0_milestone/chain_first_bad_cell_flat \
sim:/tb_stage0_milestone/scan_out_probe \
sim:/tb_stage0_milestone/i \
sim:/tb_stage0_milestone/k \
sim:/tb_stage0_milestone/fails_seen

VSM> run -all
[golden capture] response = 00 01 00 01 00 01 00 10
# PASS: zero-fault golden run is clean (0 mismatches)
[Stage 0, healthy] chain_ok = 11 (expect all 1)
[Stage 0, chain 0 cell 3 stuck] chain_ok = 10 (expect chain 0 = 0)
# first bad cell, chain 0 = 2
# PASS: Stage 0 correctly flagged the broken chain
[Stage 0, RTL-injected stuck, chain 0 cell 3] chain_ok = 10 (expect chain 0 = 0)
# PASS: RTL-injected stuck chain cell correctly flagged
[Stage 0, broken/held, chain 0 cell 3] chain_ok = 10
# PASS: broken chain cell correctly flagged as a chain anomaly (detected, same as stuck -- expected, see note above)
[Stage 0, slow, chain 1 cell 4] chain_ok = 01
# PASS: slow chain cell detected as a chain-timing anomaly (NOT proof of real at-speed-vs-reduced-speed behavior -- see note above)
# fail_valid: pattern_id=1 chain=0 cell=1
# PASS: injected SMO fault produced 1 fail-log entry
# fail_valid: pattern_id=1 chain=0 cell=2
# fail_valid: pattern_id=1 chain=0 cell=7
# PASS: injected SAI fault produced 2 fail-log entries
# EXPECTED (BEFORE): transition fault not detected under two_pattern_mode=0 -- no second capture edge to form a V1-V2 pair
# fail_valid: pattern_id=1 chain=0 cell=7
# PASS (AFTER): transition fault produced 1 fail-log entry under two_pattern_mode=1 -- the FSM extension closes the gap
** Note: $finish : C:/Users/pgkri/Downloads/tb_stage0_milestone (3).sv(434)
# Time: 6230 ns Iteration: 1 Instance: /tb_stage0_milestone
```

Fig. 3. ModelSim transcript of tb_stage0_milestone: Stage 0 chain tests and functional fault detection.

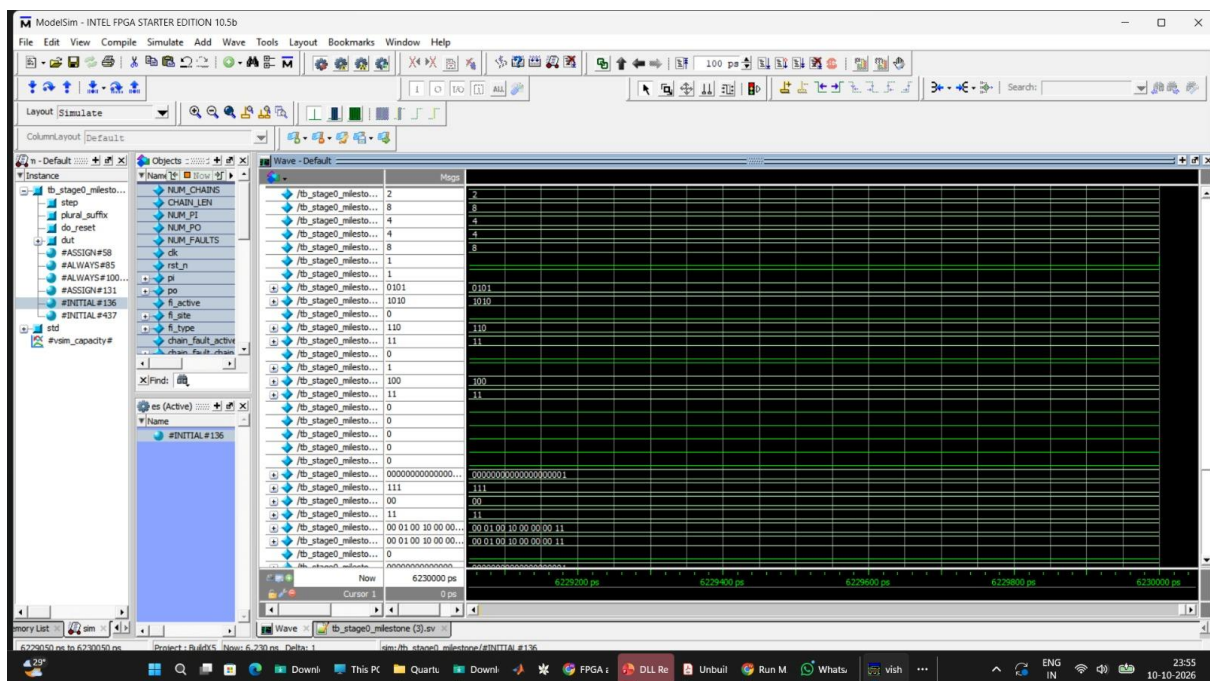


Fig. 4. ModelSim waveform of tb_stage0_milestone.

```

Transcript
File Edit View Bookmarks Window Help
Transcript
sim:/tb_stage1_milestone/po \
sim:/tb_stage1_milestone/fi_active \
sim:/tb_stage1_milestone/fi_site \
sim:/tb_stage1_milestone/fi_type \
sim:/tb_stage1_milestone/start \
sim:/tb_stage1_milestone/chain_test_mode \
sim:/tb_stage1_milestone/two_pattern_mode \
sim:/tb_stage1_milestone/busy \
sim:/tb_stage1_milestone/done \
sim:/tb_stage1_milestone/pattern_id \
sim:/tb_stage1_milestone/shift_index \
sim:/tb_stage1_milestone/shift_in_bits \
sim:/tb_stage1_milestone/golden_bits \
sim:/tb_stage1_milestone/golden_capture \
sim:/tb_stage1_milestone/new_session \
sim:/tb_stage1_milestone/fail_mask \
sim:/tb_stage1_milestone/candidate_sites \
sim:/tb_stage1_milestone/any_cell_failed \
sim:/tb_stage1_milestone/f \
sim:/tb_stage1_milestone/passes \
sim:/tb_stage1_milestone/fails
VSM> run -all
# PASS: clean pattern (no fault) candidates=11111111
# PASS: any_cell_failed=0 on clean pattern
# PASS: site0 SA0 -> unique (0) candidates=00000001
# PASS: site1 SA1 -> unique (1) candidates=00000010
# PASS: site2 SA2 -> unique (2) candidates=00000100
# PASS: site3 SA3 -> unique (3) candidates=00001000
# PASS: site4 SA4 -> ambiguous (0,1,4,6) candidates=01010011
# PASS: site5 SA5 -> ambiguous (0,1,4,6) candidates=01010011
# PASS: site6 SA6 -> ambiguous (0,1,4,6) candidates=01010011
# PASS: site7 SA7 -> ambiguous (2,3,5,7) candidates=10101100
# PASS: site8 SA8 -> masked, nothing learned candidates=11111111
# PASS: session: after site0 SA0 candidates=00000001
# PASS: session: clean pattern doesn't widen candidates=00000001
# PASS: contradictory patterns -> empty set candidates=00000000
# PASS: two-pattern run really failed (fail_mask=00010000000010001), so the guard is what's tested
# PASS: two-pattern_mode ignored by Stage 1 candidates=11111111
#
# Stage 1 summary: 15 passed, 0 failed
# ** Note: cfinish : C:/Users/pgkri/Downloads/tb_stage1_milestone (2).sv(226)
# Time: 9331 ns Iteration: 0 Instance: /tb_stage1_milestone
#
# Break in Module tb_stage1_milestone at C:/Users/pgkri/Downloads/tb_stage1_milestone (2).sv line 226
VSM 7>

```

Fig. 5. ModelSim transcript of tb_stage1_milestone: Stage 1 candidate fault sites (15 passed, 0 failed).

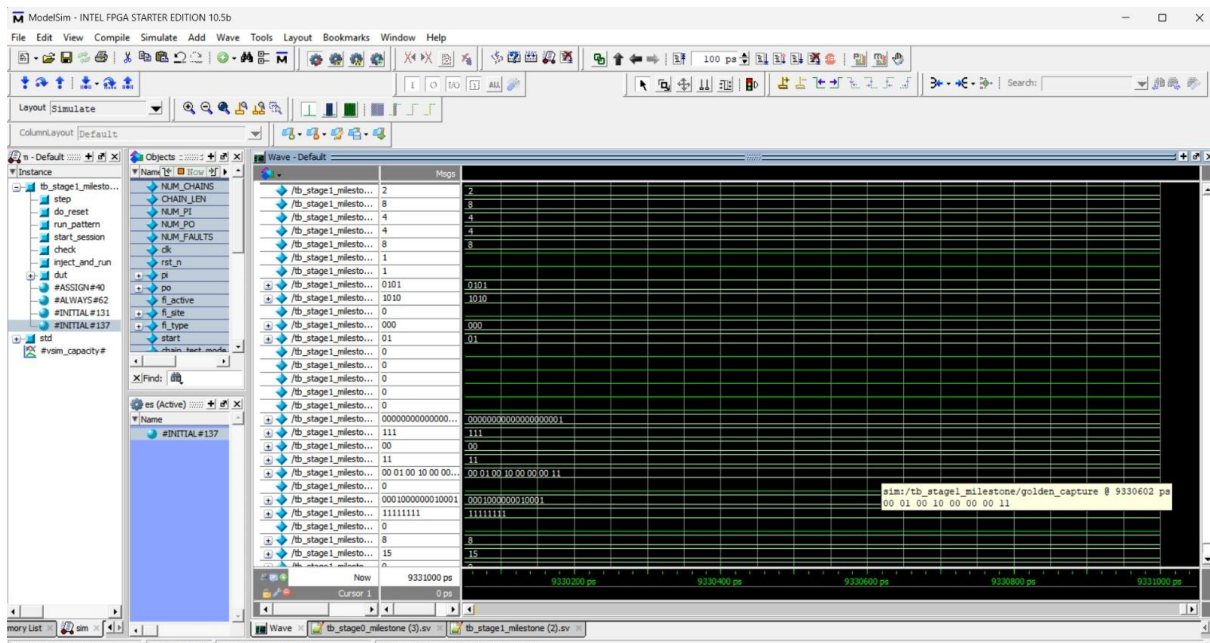


Fig. 6. ModelSim waveform of tb_stage1_milestone.

The Stage 0 results in Fig. 3 show that the fault-free golden run produced no mismatches. A healthy design gives chain_ok = 11, while a stuck cell at chain 0, cell 3 gives chain_ok = 10 with the first bad cell correctly identified; the RTL-injected stuck, broken and slow chain cells are all flagged. A stuck-at-0 fault at site 2 produced one fail-log entry and a stuck-at-1 fault at site 1 produced two. The transition fault was not detected with a single capture clock, as expected, and was detected with one fail-log entry once two_pattern_mode was enabled. The simulation completed at 6230 ns.

The Stage 1 results in Fig. 5 show that a clean pattern leaves all eight sites as candidates (candidates = 11111111). Faults at sites 0, 1, 2 and 3 are narrowed to the single correct site. Faults at sites 4 and 6 are narrowed to {0, 1, 4, 6} and a fault at site 7 to {2, 3, 5, 7}; in each case the true site is inside the set, and separating these sites is the task of the later Stage 2 fault-simulation step. A stuck-at-0 fault at site

5 is masked by the test vector, so no candidates are removed. A clean pattern never widens the set, contradictory patterns empty it, and two-pattern runs are correctly ignored. All 15 checks passed, with the simulation finishing at 9331 ns.

Hardware Implementation:

The design is targeted at the Terasic DE10-Nano development board, which carries an Intel Cyclone V SoC (5CSEBA6U23I7) combining FPGA fabric with a dual-core ARM Cortex-A9 hard processor system (HPS). In the full architecture, Stages 0 to 2 of the diagnosis engine and the circuit under test run in the FPGA fabric at the internal clock rate, while the adaptive Stage 3 diagnosis loop, the software reference model and result reporting run on the ARM processor. Placing the CUT inside the same FPGA removes pin-count and I/O-speed limits on the scan chains and allows faults to be injected at any instrumented net with a known ground truth.

The RTL of this experiment is written in synthesizable SystemVerilog and has passed open-source lint and generic synthesis checks without errors. Compilation in Quartus Prime for the Cyclone V device, pin assignment, timing analysis and on-board testing are the next steps; in the board build, the start, test-mode and fault-injection controls can be mapped to the board's push-buttons and slide switches, and the chain_ok, fail_valid and candidate-site outputs to the user LEDs.

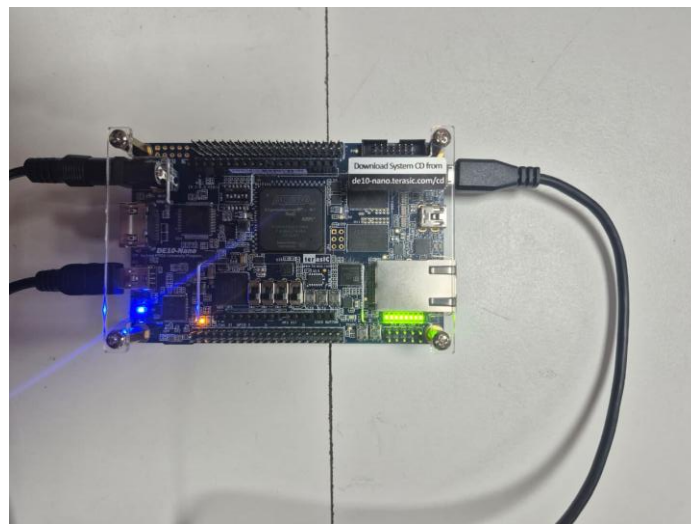


Fig. 7. Terasic DE10-Nano FPGA development board (Intel Cyclone V SoC).

Applications:

The fault diagnosis engine developed in this experiment demonstrates how scan-based testing, fault-log retention and structural fault localization can be built directly into FPGA hardware. The architecture is relevant to production testing and yield learning, where failing parts must be diagnosed down to a specific net so that the physical cause can be found quickly, and to in-field health monitoring of safety-critical electronics, such as automotive, aerospace and medical systems, where a device must detect and report its own faults during idle periods without being removed from service.

The built-in fault injection makes the same platform useful for evaluating test patterns and diagnosis algorithms against known faults, and for teaching design-for-testability concepts such as scan insertion, stuck-at and transition fault models and scan-chain integrity testing. The modular structure also allows the placeholder circuit to be replaced by standard benchmark circuits and extended with fault simulation and adaptive diagnosis stages.

Conclusion:

The experiment successfully demonstrated the first two stages of an FPGA-oriented autonomous fault diagnosis and localization engine. A scan-based test controller with chain integrity testing, two-pattern transition testing and full fail-log retention was designed in SystemVerilog, together with fault-injection hardware and a Stage 1 cone-intersection localizer. The RTL was verified in ModelSim, where the Stage 0 testbench passed all 8 checks with 1 expected outcome and the Stage 1 testbench passed all 15 checks, with stuck-at faults at four of the eight sites localized to the exact net. The work provided practical experience in design-for-testability, scan-chain design, fault modelling, finite-state-machine design and SystemVerilog verification.